

CHAPTER 05

영상의 밝기와 명암비 조절

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

5장 영상의 밝기와 명암비 조절

5.1 영상의 밝기 조절

5.2 영상의 명암비 조절

5.3 히스토그램 분석

5.2 영상의 명암비 조절

기본적인 명암비 조절 방법

- 명암비란 영상에서 가장 밝은 값과 가장 어두운 값의 차이, 명암 대비 또는 콘트라스트(contrast)라고도 함
- 영상이 전반적으로 어둡거나 또는 전반적으로 밝은 픽셀로만 구성된 경우, 명암비가 낮다고 표현함
- 반면에 밝은 영역과 어두운 영역이 골고루 섞여 있는 영상은 명암비가 높다고 말함
- 명암비가 낮은 영상은 사물의 구분이 잘 되지 않아서 전반적으로 흐릿하게 느껴짐
- 명암비가 높은 영상은 사물의 구분이 잘 되며 선명한 느낌을 줌

기본적인 명암비 조절 방법

- 그림 5-7은 서로 다른 명암비를 갖는 영상의 예임
- (a) : 명암비가 낮음, 꽃과 나뭇잎의 구분이 잘 안되고 흐릿하고 탁함
- (b) : 명암비가 높은 영상, 사물의 구분이 뚜렷하고 선명함



(a)



(b)

그림 5-7 서로 다른 명암비를 갖는 영상

기본적인 명암비 조절 방법

- 명암비 조절은 기본적으로 곱셈 연산을 사용하여 구현

$$dst(x, y) = saturate(s \cdot src(x, y))$$

- src는 입력영상, dst는 출력영상, 상수 s는 0보다 큰 양의 실수, saturate는 포화 연산 함수, x, y는 픽셀의 좌표값
- $s < 1$ 인 경우 명암비가 낮아지는 효과
- $s > 1$ 인 경우 명암비가 높아지는 효과
- $s = 2$ 인 경우 입력 픽셀값 60, 100이 출력 픽셀값 120, 200으로 변환되어 명암비는 40에서 80으로 2배 증가함
- $s = 0.5$ 인 경우 입력 픽셀값 60, 100이 출력 픽셀값 30, 50으로 변환되어 명암비는 40에서 20으로 1/2배 감소함

기본적인 명암비 조절 방법

- 아래 수식에서 $s = 0.5$ 인 경우와 $s = 2$ 인 경우의 그래프

$$dst(x, y) = saturate(s \cdot src(x, y)) \Rightarrow Y = saturate(sX)$$

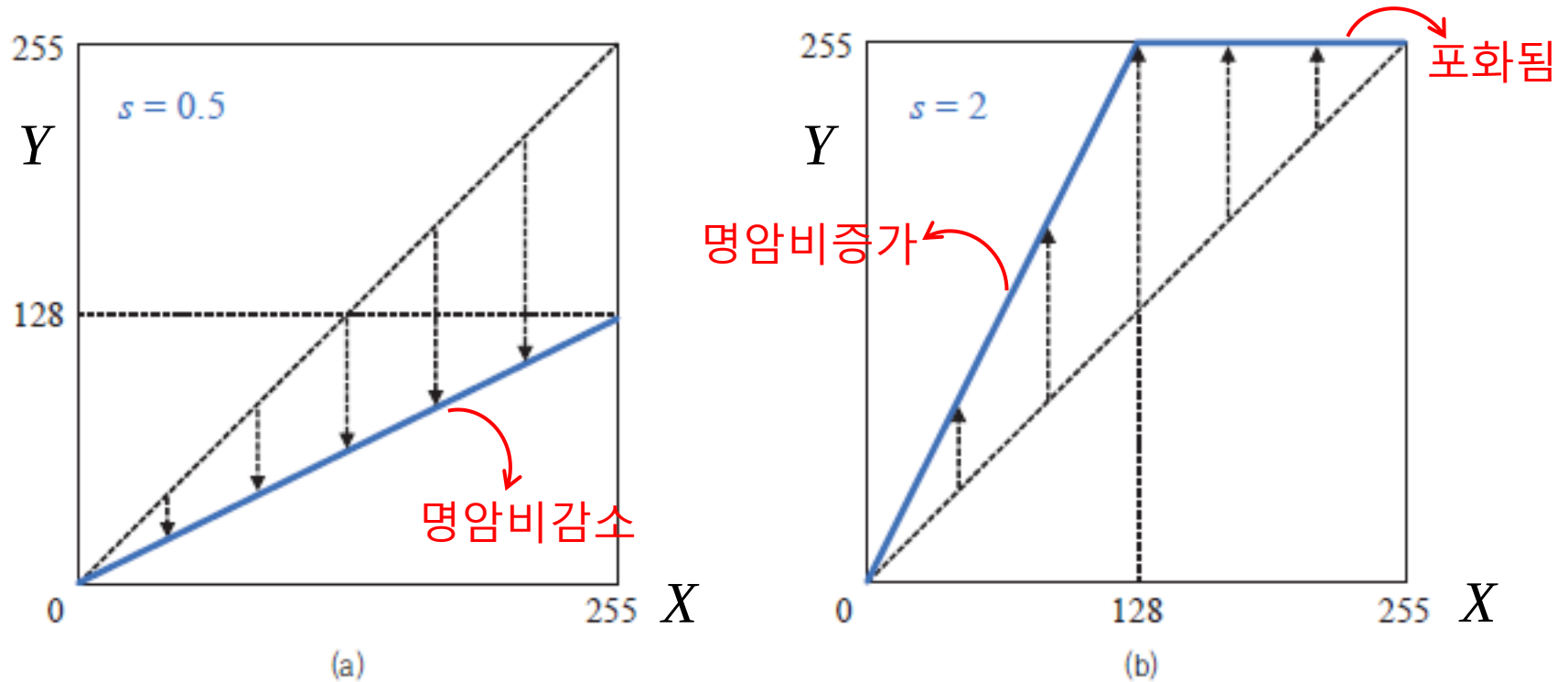


그림 5-8 기본적인 영상의 명암비 조절 함수 그래프

코드5-5 : 기본적인 명암비 조절 방법

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main(void)
{
    Mat src = imread("lenna.bmp", IMREAD_GRAYSCALE);
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1;}

    float s = 2.f;           // float형 상수
    Mat dst = s * src;       // 실수*Mat객체 -> *연산자함수(포화연산포함)

    imshow("src", src);
    imshow("dst", dst);

    waitKey();
    return 0;
}
```


코드5-5 : 기본적인 명암비 조절 방법

- 코드 5-5에서 주목해야 할 부분은 10~11행 코드임

```
float s = 2.f;  
Mat dst = s * src;
```

- OpenCV는 C/C++ 실수형 자료형(float, double)과 Mat 객체 사이의 곱셈 연산자 함수를 제공함 -> Mat 행렬의 모든 원소에 주어진 실수 값을 곱한 행렬을 반환함
- 연산자함수에서 곱셈 결과에 대한 포화 연산도 함께 수행함

코드5-5 : 기본적인 명암비 조절 방법

- dst 영상을 보면 전체적으로 픽셀 값이 포화되어 흰색으로 나타나는 영역이 너무 많으며, 이로 인해 사물의 윤곽 구분이 더 어려워짐
- 사실상 픽셀 값에 일정 상수를 단순히 곱하여 명암비를 조절하는 방식은 실전에서는 잘 사용되지 않음



그림 5-9 기본적인 영상의 명암비 증가 예제 실행 결과

효과적인 명암비 조절 방법

- 명암비를 효과적으로 높이기 위해서는 밝은 픽셀은 더욱 밝게, 어두운 픽셀은 더욱 어두워지게 변경해야 함
- 이때 픽셀 값이 밝고 어둡다는 기준을 어떻게 설정할 것인지가 명암비 조절 결과 영상의 품질 차이를 가져올 수 있음
- 그레이스케일 범위 중간값인 128을 기준으로 설정할 수도 있고, 입력 영상의 평균 밝기를 구하여 기준으로 삼을 수도 있음
- 그레이스케일 범위 중간값인 128을 기준으로 명암비를 조절하는 방법을 구현해 보자
- 명암비를 증가시키려면 입력 영상의 픽셀 값이 128보다 크면 더욱 밝게 만들고, 128보다 작으면 픽셀 값을 더 작게 만드는 방식임
- 명암비를 감소시키려면 입력 영상 픽셀 값이 128보다 크면 더 작게 만들고, 128보다 작은 픽셀 값은 더 증가시킴

효과적인 명암비 조절 방법

- 효과적인 명암비 조절 수식(입력 픽셀값 128을 기준으로 조절)

$$dst(x, y) = src(x, y) + (src(x, y) - 128) \cdot \alpha, \quad \alpha \geq -1 \text{인 실수}$$

- 이 수식은 항상 (128, 128) 좌표를 지나가고, α 에 의해 기울기가 변경되는 직선의 방정식임
- $-1 \leq \alpha \leq 0$ 이면 기울기가 0부터 1사이의 직선이 되며, 이는 명암비를 감소시키는 변환 함수임
- $\alpha > 0$ 이면 기울기가 1보다 큰 직선의 방정식이며, 이는 명암비를 증가시키는 변환 함수임
- 결과 영상의 픽셀 값은 0보다 작거나 255보다 커지는 경우가 발생할 수 있으므로 포화 연산도 함께 수행해야 함

$$dst(x, y) = \text{saturnate} \left(src(x, y) + (src(x, y) - 128) \cdot \alpha \right)$$

효과적인 명암비 조절 방법

- $\alpha = -0.5$ 인 경우와 $\alpha = 1.0$ 인 경우의 함수 그래프

$$Y = \text{saturation}(X + (X - 128)\alpha) \Rightarrow Y = \text{saturation}((\alpha + 1)X - 128\alpha)$$

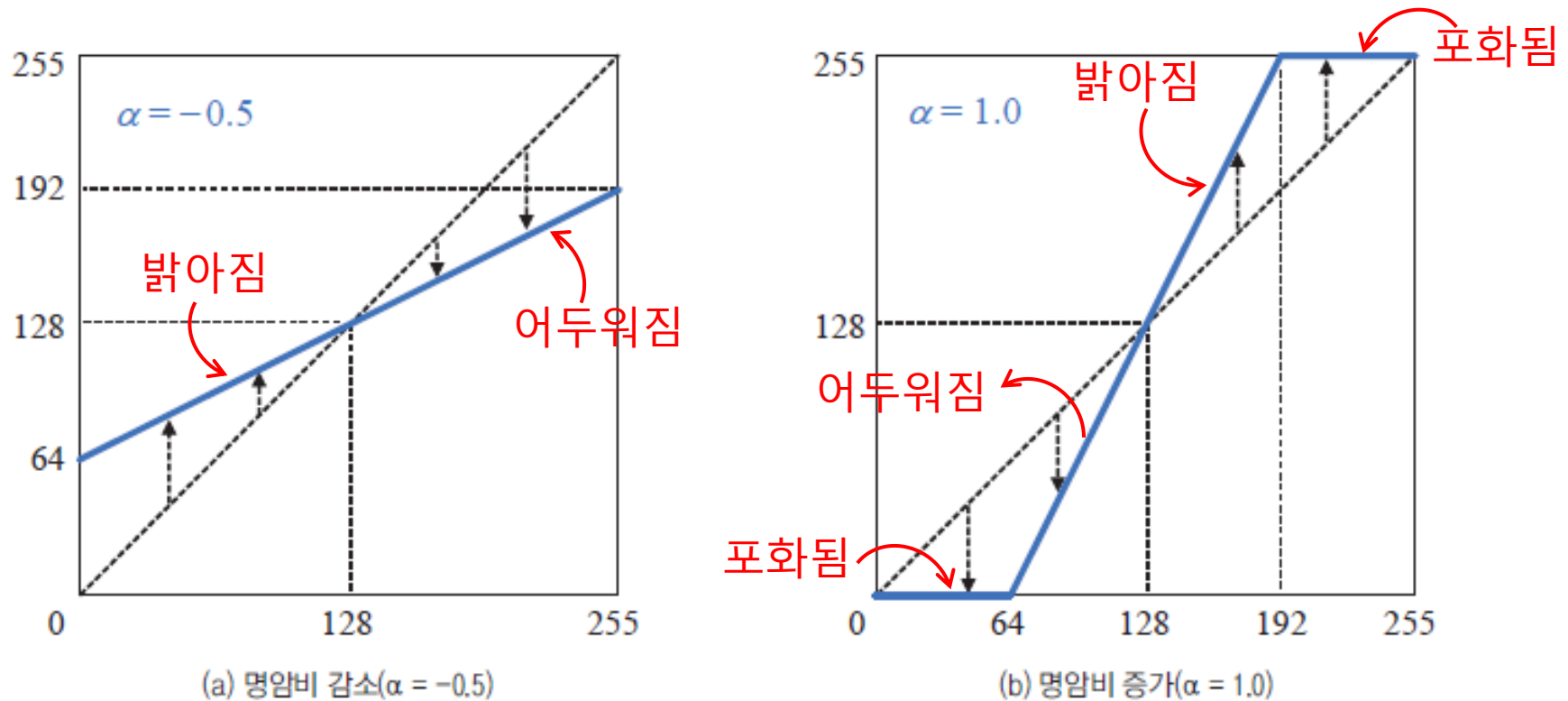


그림 5-10 효과적인 영상의 명암비 조절 함수 그래프

코드5-6 : 효과적인 명암비 조절 방법

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main(void)
{
    Mat src = imread("lenna.bmp", IMREAD_GRAYSCALE);
    if (src.empty()) {cerr << "Image load failed!" << endl; return -1;}

    float alpha = 1.f;
    Mat dst = src + (src - 128) * alpha;    /*,+, -연산자함수(포화연산포함)

    imshow("src", src);
    imshow("dst", dst);
    waitKey();
    return 0;
}
```

코드5-6 : 효과적인 명암비 조절 방법

- 그림 5-9에서 보았던 기본적인 명암비 증가 예제보다 그림 5-11에 나타난 결과 화면이 좀 더 자연스럽게 명암비가 증가됨

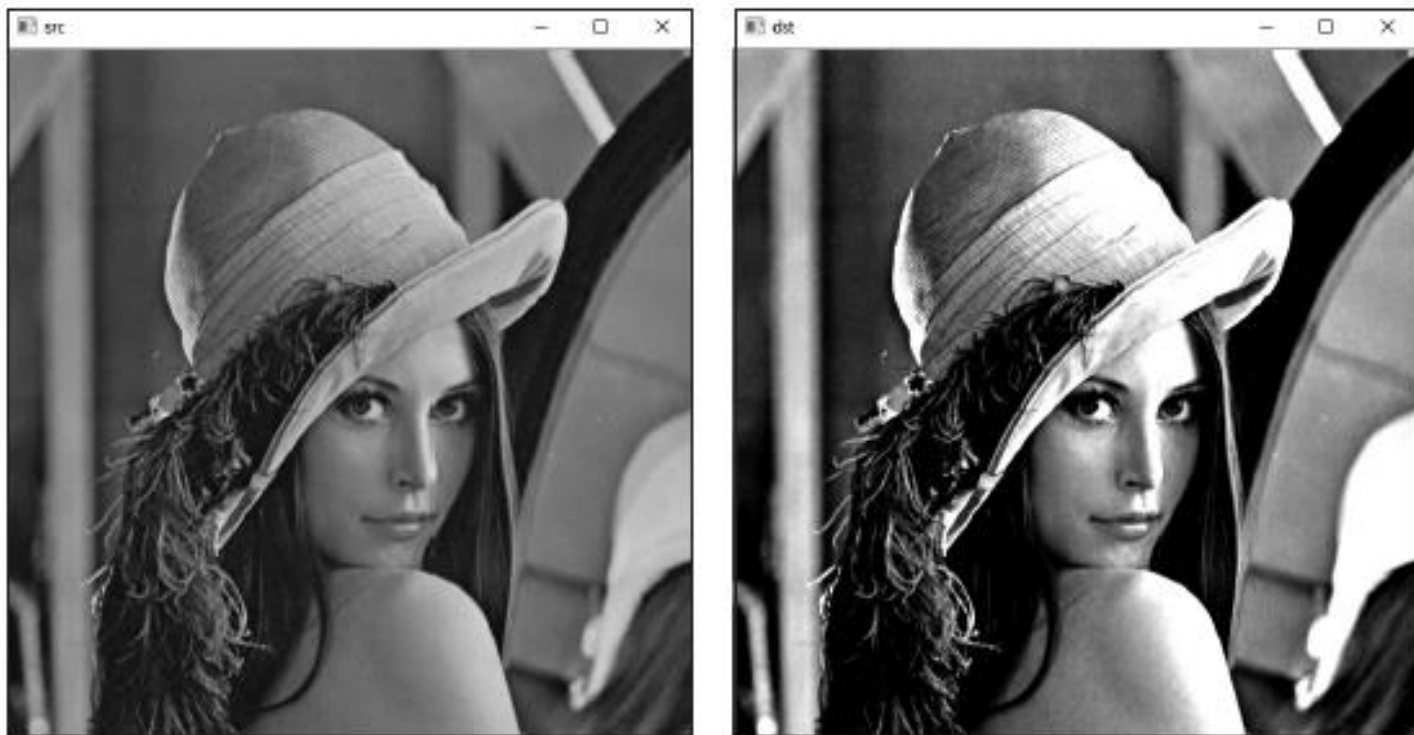


그림 5-11 효과적인 영상의 명암비 증가 예제 실행 결과

실습과제1

- 코드 5-5에서 연산자 함수를 사용하지 말고 픽셀값을 직접 참조하는 방식으로 같은 결과를 얻도록 수정하라.
- 204페이지 코드 5-3을 참고할 것.

실습과제2

- 코드 5-6에서 연산자 함수를 사용하지 말고 픽셀값을 직접 참조하는 방식으로 같은 결과를 얻도록 수정하라.
- 204페이지 코드 5-3을 참고할 것.

실습과제3

- 코드5-6에서 α 가 너무 크면 어떻게 되는가? 실행결과를 첨부하고 이유를 설명하라.
- 코드5-6에서 α 가 너무 작으면 어떻게 되는가? 실행결과를 첨부하고 이유를 설명하라.

실습과제4

- 코드5-6은 입력 영상의 픽셀값을 128을 기준으로 구분하여 명암비를 개선한다. 여기서 128을 입력영상 픽셀값의 평균값으로 변경하여 명암비 개선을 수행하라.
- 영상의 픽셀값의 평균값을 구할때는 함수 `mean()`을 이용하라.
- `Scalar avg = mean(img);` -> `img`가 그레이 영상이면 `avg[0]`에 평균값이 저장됨
- 코드5-6의 결과와 비교하라. 픽셀의 평균값이 128과 비슷하다면 결과도 비슷할 것이다.

과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.